专业：微电子科学与工程

姓名：

学号：

浙江大学 实验报告

课程名称：电子电路设计实验 I　　指导老师：屈民军、唐奕　　成绩：

实验名称：音乐播放实验　　实验类型：设计类实验

一、实验目的

(1) 掌握音符产生的方法，了解 DDS 技术的应用。

(2) 了解音频解码的应用。

(3) 掌握系统“自顶而下”的数字系统设计方法。

二、实验任务与要求

1、设计并仿真一个 DDS 正弦信号发生器，要求：

(1) 采样频率：$f_c = 48\text{kHz}$

(2) 正弦信号频率范围：$20\text{Hz} \sim 20\text{kHz}$

(3) 正弦信号序列宽度为 16 位，包括 1 位符号位。

2、设计一个音乐播放器，要求：

(1) 可以播放四首乐曲，设置 play/pause_button、next_button、reset 三个按键。按 play/pause_button 键，音乐在播放与暂停之间切换；按 next_button 键播放下一首乐曲。

(2)LED0 指示播放状态（播放时点亮），LED2 与 LED3 指示当前乐曲序号。

三、实验原理

3.1 DDS 正弦信号发生器

1、DDS 实验基本原理

要在数字域产生正弦信号，可以用一个存储器（ROM/RAM）存储一张正弦表；然后将存于表中的正弦样品取出，经数模转换器 D/A，形成模拟量波形。若要实时改变输出信号的频率，可以通过改变查表寻址的频率或寻址的步长来实现。DDS 技术即采用了后面这种方法，步长为对数字波形查表的相位增量，输出正弦频率与相位增量成线性关系。

DDS 的基本原理框图如图 1 所示，由相位累加器、正弦查询表（Sine ROM）、D/A 转换器和低通滤波器组成：

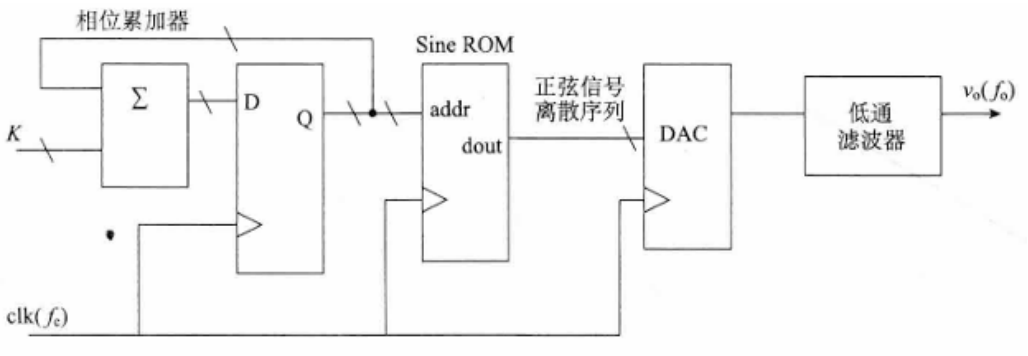


图 1: DDS 的基本原理框图

Sine ROM 中存放一个完整的正弦信号样品，其由如下的映射关系构成：

$$S(i) = (2^{n-1} - 1) \times \sin\left(\frac{2\pi i}{2^n}\right), \quad i = 0, 1, 2, \ldots, 2^{n-1}$$

式中，m 为 Sine ROM 地址线位数，n 为 ROM 数据线宽度，$S(i)$ 的数据形式为补码。

f_c 为取样时钟 clk 的频率，K 为相位增量（也称频率控制字），输出正弦信号的频率 f_o 由 f_c 和 K 共同决定，即：

$$f_o = \frac{K \times f_c}{2^m}$$

可以看出，正弦信号的频率 f_0 与相位增量 K 呈正比关系。相位累加器的位数由 m 位整数和 p 位小数组成，其高 m 位整数部分作为 Sine ROM 的地址。

在实际应用当中，DDS 的最高输出频率由允许输出的杂散电平决定，一般取值为 $f_0 \leq 40\% \times f_c$，因此 K 的最大值一般为 $40\% \times 2^{m-1}$。除此之外，DDS 可以很容易实现正弦信号与余弦信号正交两路输出，只需用相位累加器的输出同时驱动固化有正弦信号波形的 Sine ROM 和余弦信号波形的 Cos ROM，并各自经数模转换器和低通滤波器输出即可。另外，DDS 也很容易实现调幅和调频，其原理框图分别如图 2 和图 3 所示：

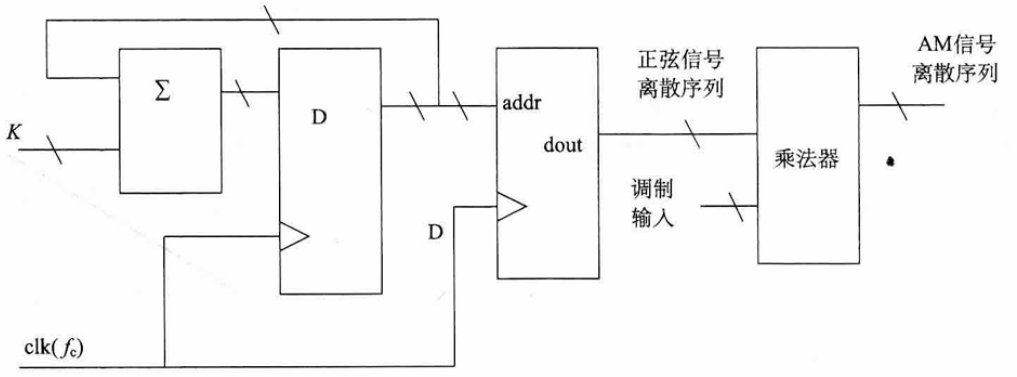


图 2: DDS 实现调幅原理框图

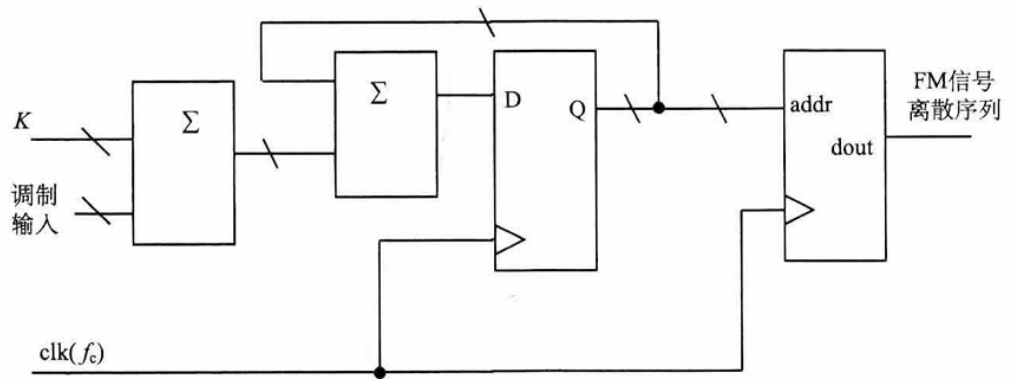


图 3: DDS 实现调频原理框图

2、DDS 实验设计原理

根据 DDS 输出信号的最低频率要求，有：

$$f_o(\min) = \frac{f_c}{2^m} \leq 20\text{Hz}$$

代入 $f_c = 48\text{kHz}$，可得 $m = 12$，但为了得到更准确的正弦信号频率，相位累加器位数会增加 10 位小数。所以，相位累加器为 22 位累加器，其高 12 位为 Sine ROM 的地址。

因为 $m = 12$，所以存储一个完整周期的正弦信号样品就需要 $2^{12} \times 16$ bits 的 ROM。

但由于正弦波形的对称性，如图 4 所示，可以将正弦波形分为四个区域，如此只需要在 Sine ROM 中存储四分之一的正弦信号样品（0 区）即可。此时 Sine ROM 的容量可减少为 $2^{10} \times 16$ bits，即 10 位地址，存储四分之一的正弦信号样品（共 1024 个）。

需要注意的是，四分之一周期的正弦信号样品未给出 90° 的样品值，因此在 ROM 地址为 1024（即 90°）时可取地址为 1023 的值（实际上地址为 1023 时，正弦信号样品已达最大值）。

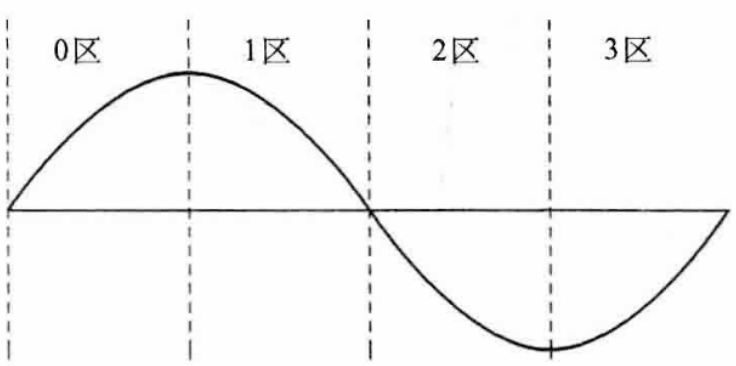


图 4: 正弦信号波形

因为必须利用 1/4 周期的 1024 个样品复制出一个完整正弦周期的 4096 个样品，所以对 DDS 结构要进行必要处理，优化后的 DDS 结构如图 5 所示：

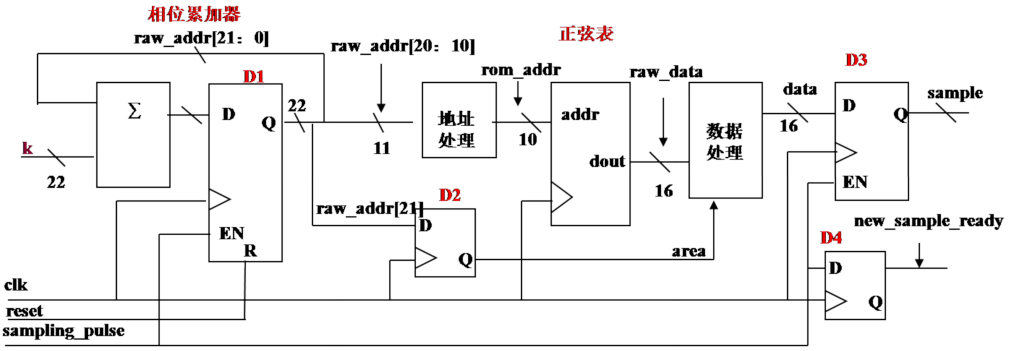


图 5: 优化后的 DDS 结构

图中，sampling_pulse 为采样脉冲，一般情况下，其频率应低于时钟 clk 频率，而宽度必须为一个时钟 clk 周期。若采样脉冲与时钟信号为同一个信号，可将 sampling_pulse 接高电平即可。相位累加得到的 22 位原始地址 $raw_addr[21:0]$，整数部分 $raw_addr[21:10]$ 即为完整周期正弦信号样品的地址，其中高两位地址 $raw_addr[21:20]$ 可区分正弦的四个区域。由于 sine_rom 只保存了四分之一周期的 1024 个样品，所以 $raw_addr[21:10]$ 不能直接作为 sine_rom 地址，必须进行必要处理，其处理方法如图 6 所示：

3.2 音乐播放器

1、音乐播放器顶层设计

根据实验任务可将系统划分为时钟管理模块 (DCM)、按键处理、主控制器、乐曲读取、音符播放 (note_player)、同步化电路、节拍基准产生器和音频编解码接口电路等子模块，如

正弦区域	Sine ROM 地址	正弦样品 data	备注
0	raw_addr[19:10]	raw_data[15:0]	
1	当 raw_addr[20:10]=1024 时，rom_addr 取 1023，其他情况取~raw_addr[19:10]+1	raw_data[15:0]	地址镜像翻转
2	raw_addr[19:10]	~raw_data[15:0]+1	数据取反
3	当 raw_addr[20:10]=1024 时，rom_addr 取 1023，其他情况取~raw_addr[19:10]+1	~raw_data[15:0]+1	地址镜像翻转 数据取反

图 6: sine_rom 地址和数据处理方法

图 7 所示。

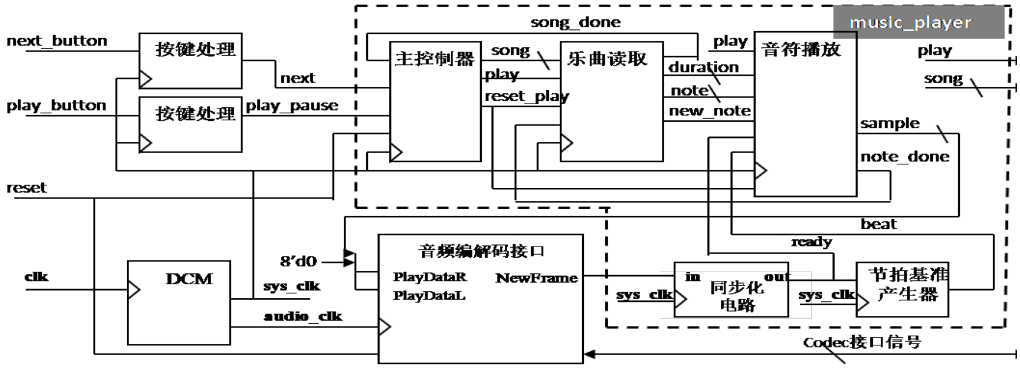


图 7: 系统的顶层框图

各主要子模块作用如下：

时钟管理模块 (DCM) 产生 100MHz 的系统时钟 sys_clk 和 12.5MHz 的音频时钟 audio_clk。

主控制器 (mcu) 模块接收按键信息，通知 song_reader 模块是否要播放 (play) 及播放哪首乐曲 (song)。

乐曲读取 (song_reader) 模块根据 mcu 模块的要求，逐个取出音符信息 (note, duration) 送给 note_player 模块播放，当一首乐曲播放完毕，回复 mcu 模块乐曲播放结束信号 (song_done)。

音符播放接收到播放的音符，在音符的持续时间以内，以 48kHz 速率送出该音符的正弦波样品给音频编解码接口模块。当一个音符播放结束，向 song_reader 模块发送一个 note_done 脉冲获取新的音符。

音频编解码接口模块负责将音符的正弦波样品转换为串行输出并发送给音频编解码芯片 ADAU1761。音频编解码芯片 ADAU1761 接收正弦波样品，再进行 AD 转换并放大，最后送至扬声器播放。注意，note_player 模块产生的正弦波样品为 16 位二进制，需在低位加 8 个

0 后送入音频编解码接口模块。

由于音频编解码模块与系统使用不同时钟，因此需要同步化电路协调两部分电路。

节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号 (beat)，而 ready 信号频率为 48kHz，因此，节拍基准产生器即为分频比为 1000 的分频器。而按键处理模块完成输入同步化、防颤动和脉宽变换等功能。

2、音乐播放器设计原理

①主控制器模块 mcu 的设计

主控制模块 mcu 有响应按键信息、控制系统播放两大任务，下表为其端口含义：

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play_pause	Input	来自按键处理模块的“播放/暂停”控制信号，一个时钟周期宽度的脉冲
next	Input	来自按键处理模块的“下一曲” 控制信号，一个时钟周期宽度的脉冲
play	Output	输出控制信号，高电平表示播放，控制 song_reader 模块是否要播放
reset_play	Output	时钟周期宽度的高电平复位脉冲 reset_play，用于同时复位模块 song_reader 和 note_player
song_done	Input	song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示一曲播放结束
song[1:0]	Output	当前播放乐曲的序号

图 8: 主控制器模块 mcu 的端口含义

根据设计要求，模块 mcu 的原理框图如图 9 所示。图中的 2 位二进制计数器用来计算乐曲序号 (song)：

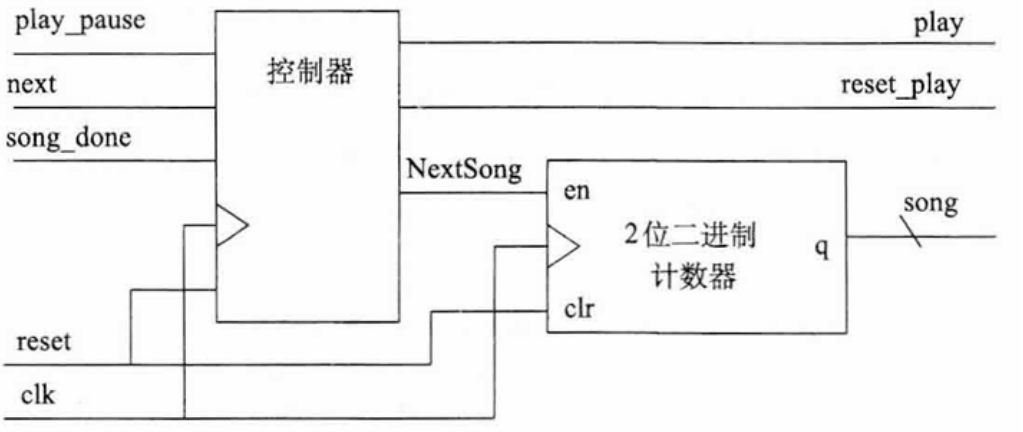


图 9: mcu 的结构框图

其控制器的工作流程图如图 10 所示：

②乐曲读取模块 song_reader 的设计

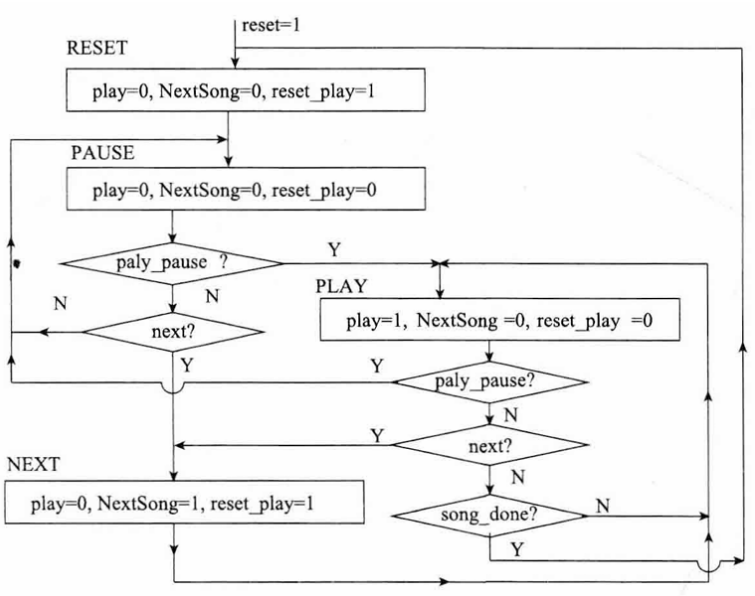


图 10: mcu 控制器的算法流程图

乐曲读取模块 song_reader 的任务有：

(1) 根据 mcu 模块的要求，选择播放乐曲；

(2) 响应 note_player 模块请求，从 song_rom 中逐个取出音符 {note, duration} 送给 note_player 模块播放；

(3) 判断乐曲是否播放完毕，若播放完毕，则回复 mcu 模块应答信号。

根据 song_reader 模块的任务要求，其需包含图 11 所示的输入、输出端口：

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play	Input	来自 mcu 的控制信号，高电平要求播放
song[1:0]	Input	来自 mcu 的控制信号，当前播放乐曲的序号
note_done	Input	即模块 note_player 的应答信号，一个时钟周期宽度的脉冲，表示一个音符播放结束并索取新音符
song_done	Output	给 mcu 的应答信号，当乐曲播放结束，输出一个时钟周期宽度的脉冲，表示乐曲播放结束
note[5:0]	Output	音符标记
duration[5:0]	Output	音符的持续时间
new_note	Output	给模块 note_playe 的控制信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放

图 11: 乐曲读取模块 song_reader 的端口含义

song_rom 是一个只读存储器，用来存放乐曲，容量为 $2^7 \times 12$ bits。其中共有放有四首乐曲，每首乐曲占用 $2^5 \times 12$ bits 空间，即每首乐曲最长由 32 个音符组成。因此，song_rom 高 2 位地址决定哪首乐曲，而低 5 位地址决定这首乐曲的哪个音符。song_rom 每个地址存

放一个音符信息，音符信息由 12 位二进制组成，高 6 位表示音符标记 note，低 6 位表示音长 duration。

根据 song_reader 模块的功能及 song_rom 结构，可画出图 12 所示的结构框图，控制器主要负责接收 mcu 模块与 note_player 模块的控制信号，并做出响应。

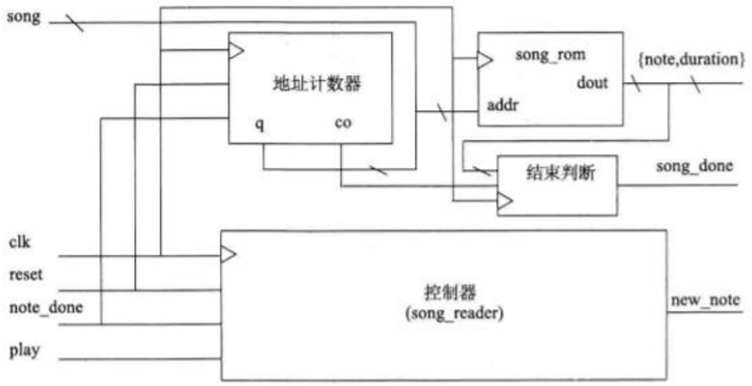


图 12: song_reader 的结构框图

控制器算法流程图如图 13 所示：

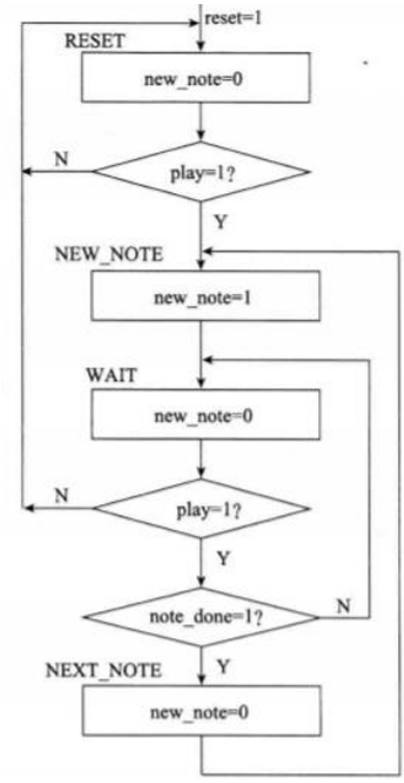


图 13: song_reader 控制器的算法流程图

③音符播放模块 note_player 的设计

音符播放模块 note_player 是本实验的核心模块，其主要任务包括：

(1) 从 song_reader 模块接收需播放的音符 {note, duration}；

(2) 根据 note 值找出 DDS 的相位增量 k；

(3) 以 48kHz 速率从 Sine ROM 取出正弦样品送给音频编解码器接口模块；

(4) 当一个音符播放完毕，向 song_reader 模块获取新的音符。

根据 note_player 模块的任务，可进一步划分功能单元，如图 14 所示，图中 FreqROM 为只读存储器，完成音符标记 note 与 DDS 模块的相位增量 k 查找表关系。

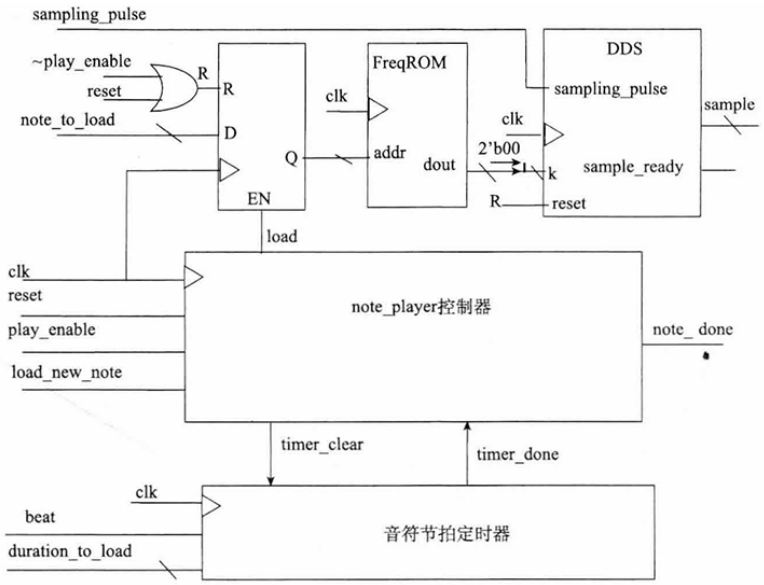


图 14: note_player 的结构框图

下图为 note_player 模块的端口含义：

引脚名称	I/O	引脚说明
clk	Input	系统时钟信号，外接 sys_clk
reset	Input	复位信号，高电平有效，外接 mcu 模块的 reset_play
play_enable	Input	来自 mcu 模块的 play 信号，高电平表示播放
note_to_load[5:0]	Input	来自 song_reader 模块的音符标记 note，表示需播放的音符
duration_to_load[5:0]	Input	来自 song_reader 模块的音符持续时间 duration，表示需播放音符的音长
load_new_note	Input	来自 song_reader 模块的 new_note 信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放
note _ done	Output	给 song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示音符播放完毕
sampling_pulse	Input	来自同步化电路模块的 ready 信号，频率 48kHz，一个时钟周期宽度的高电平脉冲，表示索取新的正弦样品
beat	Input	定时基准信号，频率为 48Hz 脉冲，一个时钟周期宽度的高电平脉冲
sample [15:0]	Output	正弦样品输出

图 15: note_player 的端口含义

note_player 控制器负责与 song_reader 模块接口，读取音符信息，并根据音符信息从

Frequency ROM 中读取相应相位增量 K 送给 DDS 子模块。另外，note_player 控制器还需要控制音符播放时间，其算法流程如图 16 所示：

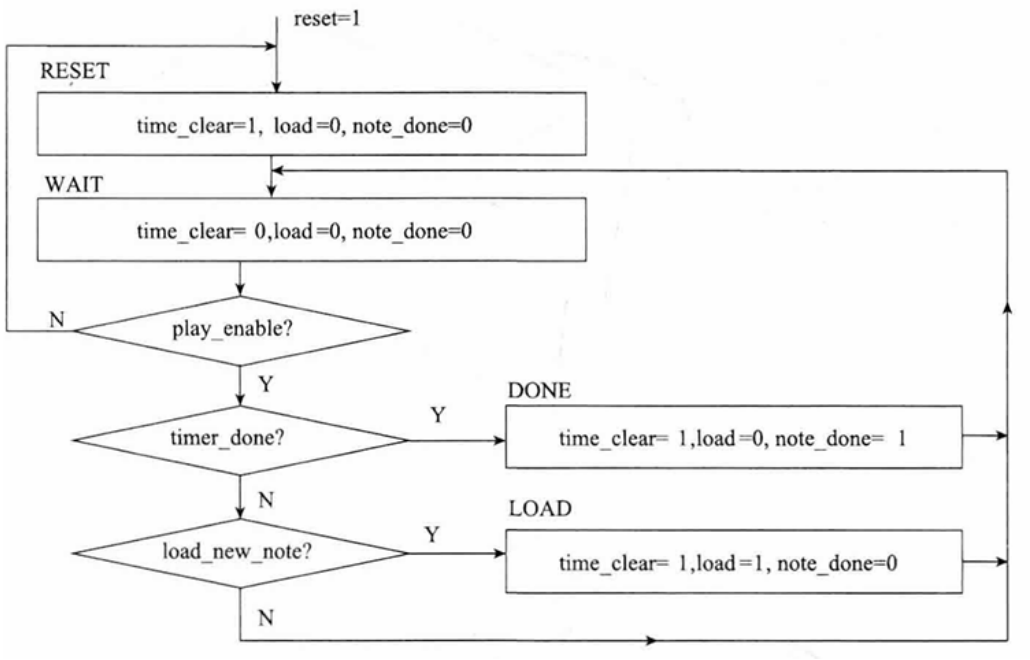


图 16: note_player 控制器的算法流程图

音符定时器为 6 位二进制计数器，beat、timer_clear 分别为使能、清 0 信号，均为高电平有效。定时时间由音长信号 duration_to_load 决定，即 duration_to_load-1 个 beat 周期，timer_done 为定时结束标志。

子模块 DDS 的功能就是利用 DDS 技术产生正弦样品。需要注意的是，DDS 模块的输入 k 为 22 位二进制，因此，Frequency ROM 输出的 20 位相位增量需要高位加两个 0 后再接入 DDS。

④同步化电路

由于音频编解码接口模块和其他模块采用不同的时钟，因此两者之间的控制及应答信号须进行同步化处理。本例中音频编解码接口模块的输出信号 NewFrame 的脉冲宽度为一个 audio_clk 时钟周期，需通过同步化处理，产生与 sys_clk 同步且脉冲宽度为一个 sys_clk 时钟周期的信号 ready。其由同步器和脉冲宽度变换电路组成，电路结果如图所示：

⑤时钟管理模块（DCM）

IP 内核时钟管理模块的输入时钟 clk 频率为 100MHz，产生 100MHz 的系统时钟和 12.50MHz 的音频时钟。

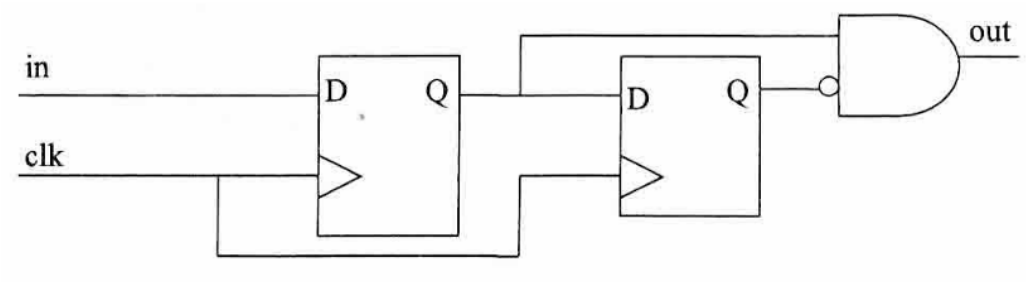


图 17: 同步化电路的结构框图

四、实验仪器设备

①装有 Vivado 和 ModelSim SE 软件的计算机；

②Nexys Video Artix-7 FPGA 多媒体音视频智能互联开发系统；

③耳机

五、实验内容

(1) 从网站下载提供的文件；

(2) 编写 DDS 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证；

(3) 编写 mcu 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证；

(4) 编写 song_reader 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证；

(5) 编写 note_player 模块的 Verilog HDL 代码，并用 ModelSim 仿真验证；

(6) 编写 1000 分频器模块的 Verilog HDL 代码及其测试代码，并用 ModelSim 仿真验证；

(7) 编写同步化电路的 Verilog HDL 代码及其测试代码，并用 ModelSim 仿真验证；

(8) 编写次顶层 music_player 模块的 Verilog HDL 代码及其测试代码，并用 ModelSim 仿真验证；

(9) 新建 music_player 的 Vivado 工程，编写顶层 music_player_top 模块的 Verilog HDL 代码，生成符合要求的 DCM 内核，添加需要的文件，对工程进行综合、约束、实现，并下载工程文件到 Nexys Video 开发板中；

(10) 将耳机接入实验开发板音频输出插座，操作 reset（中间按钮）、play/pause（右边按钮）、next（下面按钮）三个按键，试听耳机中的乐曲并观察实验板上指示灯变化情况，验证设计结果是否正确。

六、实验过程

1. DDS 模块

1.1 Verilog HDL 代码设计

DDS 模块由顶层模块 dds.v 和子模块加法器 full_adder.v、D 触发器 dffre.v 组成。

顶层模块代码如下：

```
module dds (clk, reset, sampling_pulse, k, sample, new_sample_ready);
    input clk, reset, sampling_pulse;
    input[21:0] k;                  //相位增量
    output[15:0] sample;            //sample信号为正弦序列信号
    output new_sample_ready;

    wire[21:0] raw_addr;            //地址处理输入
    wire[9:0] rom_addr;             //地址处理输出
    wire[15:0] raw_data, data;      //数据处理
    wire[21:0] sum;                 //加法器结果
    wire area;                      //区域

    //加法器实例
    full_adder adder0(.a(k), .b(raw_addr), .s(sum), .co());//进位输出空脚
    //D触发器实例
    dffre #(.n(22)) dff0(.d(sum), .en(sampling_pulse), .r(reset), .clk(
       clk), .q(raw_addr));
    dffre #(.n(1)) dff1(.d(raw_addr[21]), .en(1), .r(0), .clk(clk), .q(
       area));
    dffre #(.n(16)) dff2(.d(data), .en(sampling_pulse), .r(0), .clk(clk),
        .q(sample));
    dffre #(.n(1)) dff3(.d(sampling_pulse), .en(1), .r(0), .clk(clk), .q(
       new_sample_ready));
    //地址处理
    assign rom_addr[9:0] = raw_addr[20] ? ((raw_addr[20:10] == 1024) ?
       1023 : (~raw_addr[19:10]+1)) : raw_addr[19:10];
    //正弦查找表
    sine_rom rom0(.clk(clk), .addr(rom_addr), .dout(raw_data));
    //数据处理
    assign data[15:0] = area ? (~raw_data[15:0]+1) : raw_data[15:0];
endmodule
```

1.2 仿真结果

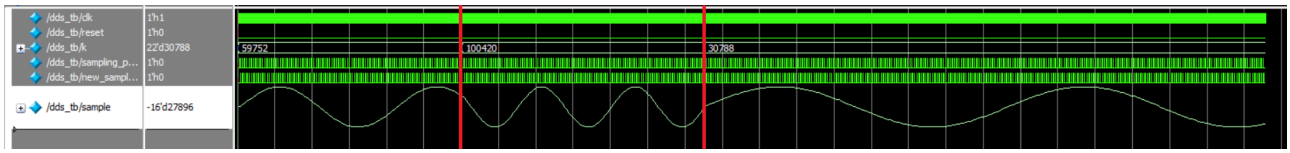


图 18: DDS 仿真结果

分析：如图所示，当 k 值变大时，正弦输出信号 sample 的频率也变大；当 k 值变小时，正弦输出信号 sample 的频率也变小；k 值发生变化的点也即频率发生变化的转折点（如红线标注处所示），符合设计要求。

2. mcu 模块

2.1 Verilog HDL 代码设计

mcu 模块由顶层模块 mcu.v 和子模块控制器 mcu_ctrl.v、D 触发器 dffre.v 组成。

顶层模块代码如下：

```
module mcu (clk, reset, play_pause, next, song_done, play, reset_play,
    song);
        input clk, reset;
        input play_pause;           //play_pause按钮
        input next;                 //next按钮
        input song_done;            //播放结束信号
        output play;
        output reset_play;          //脉冲复位
        output[1:0] song;           //乐曲序号

        wire nextsong;              //控制器输出

        //控制器实例
        mcu_ctrl ctrl0(.clk(clk), .reset(reset), .play_pause(play_pause),
            .next(next), .song_done(song_done), .play(play), .reset_play(
           reset_play), .nextsong(nextsong));

        //2位二进制计数器实例
        counter_n #(.n(4), .counter_bits(2)) song_cnt(.clk(clk), .en(
           nextsong), .r(reset), .q(song), .co());
endmodule
```

控制器模块 mcu_ctrl.v 的代码如下：

```
module mcu_ctrl(clk, reset, play_pause, next, song_done, play, reset_play, nextsong);
        input clk, reset;
        input play_pause;                       //play_pause按钮
        input next;                             //next按钮
        input song_done;                        //乐曲播放结束
        output reg play;
        output reg reset_play;                  //脉冲复位
        output reg nextsong;                    //控制器输出

        parameter RESET = 0, PAUSE = 1, PLAY = 2, NEXT = 3; //状态编码
        reg[1:0] state, nextstate; //现态与次态

        //状态寄存器
        always @ (posedge clk or posedge reset) begin
                if (reset)
                        state <= RESET;
                else
                        state <= nextstate;
        end

        //次态判断
        always @ (*) begin
                //默认保持当前状态
                nextstate = state;
                case (state)
                        RESET:
                                nextstate = PAUSE; //复位后无条件进入PAUSE
                        PAUSE: begin
                                if (play_pause)
                                        nextstate = PLAY;
                                else if (next)
                                        nextstate = NEXT;
                        end
                        PLAY: begin
                                if (play_pause)
                                        nextstate = PAUSE;
                                else if (next)
```

```
                                        nextstate = NEXT;
                                else if (song_done)
                                        nextstate = RESET;
                        end
                        NEXT:
                                nextstate = PLAY; //切歌后无条件自动播放
                        default:
                                nextstate = RESET;
                endcase
        end

        //输出逻辑
        always @ (*) begin
                //默认输出为0，避免产生锁存器
                play = 1'b0;
                nextsong = 1'b0;
                reset_play = 1'b0;
                case (state)
                        RESET: begin
                                reset_play = 1'b1;
                        end
                        PAUSE: begin
                        end
                        PLAY: begin
                                play = 1'b1;
                        end
                        NEXT: begin
                                nextsong = 1'b1;
                                reset_play = 1'b1;
                        end
                        default: begin
                                play = 1'b0;
                                nextsong = 1'b0;
                                reset_play = 1'b0;
                        end
                endcase
        end
endmodule
```

2.2 仿真结果

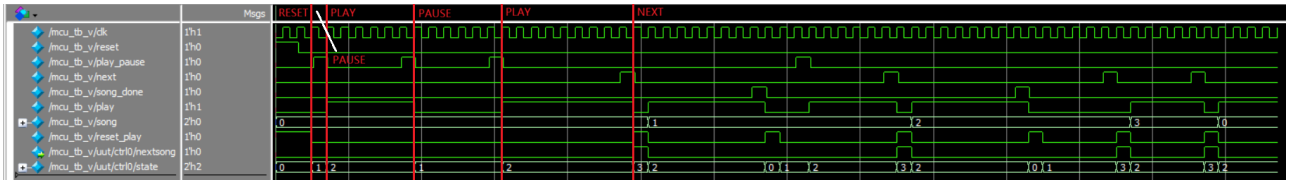


图 19: mcu 仿真结果

分析：如图所示，初始状态为 RESET（状态编码为 0），此时 reset = 1，play = 0，nextsong = 0，reset_play = 1；

在图示从左往右第一条直线处，reset = 0 时，时钟上升沿到达后，状态转移到 PAUSE（状态编码为 1），此时 play = 0，nextsong = 0，reset_play = 0；

在第二、四条直线处，reset = 0，play_pause = 1，时钟上升沿到达后，状态转移到 PLAY（状态编码为 2），此时 play = 1，nextsong = 0，reset_play = 0；

在第三条直线处，reset = 0，play_pause = 1，时钟上升沿到达后，状态转移到 PAUSE（状态编码为 1），此时 play = 0，nextsong = 0，reset_play = 0；

在第五条直线处，reset = 0，play_pause = 0，next = 1，时钟上升沿到达后，状态转移到 NEXT（状态编码为 3），此时 play = 0，nextsong = 1，reset_play = 1；

综上所述，mcu 模块的仿真结果符合设计要求。

3. song_reader 模块

3.1 Verilog HDL 代码设计

song_reader 模块由顶层模块 song_reader.v 和子模块控制器 song_reader_ctrl.v、地址计数器模块 counter_n、歌曲存储模块 song_rom 和结束判断模块 song_over 组成。其中地址计数器模块 counter_n 即为 n 位二进制计数器，其代码可直接参考课程文件中 lab1_2_FlowingWaterLigh 工程中给出的示范代码；结束判断模块 song_over 采用状态机方法实现。

顶层模块代码如下：

```
1  module song_reader(clk, reset, play, song, note_done, song_done, note,
       duration, new_note);
2          input clk, reset;
3          input play;                 //来自mcu的控制信号，高电平代表播放
4          input note_done;            //表示一个音符播放结束
5          input [1:0] song;           //当前乐曲序号
6          output song_done;           //输出给mcu的播放结束信号
7          output new_note;            //表示需播放新的音符
8          output [5:0] note;          //音符标记
```

```
        output [5:0] duration;  //音符持续时间

        wire [4:0] q;           //乐曲音符地址
        wire co;                //地址计数器进位

        //地址计数器实例，每播放完一个音符地址加1
        counter_n #(.n(32), .counter_bits(5)) addr_cnt(.clk(clk), .en(
            note_done), .r(reset), .q(q), .co(co));
        //只读存储器实例，地址为7位，其中高2位为歌曲序号，低5位为音符地址
        song_rom rom0(.clk(clk), .dout({note, duration}), .addr({song, q
            }));
        //结束判断模块实例
        song_over over0(.clk(clk), .reset(reset), .ci(co), .din(duration)
            , .dout(song_done));
        //控制器模块实例
        song_reader_ctrl ctrl0(.clk(clk), .reset(reset), .play(play), .
            note_done(note_done), .new_note(new_note));
endmodule
```

控制器模块 song_reader_ctrl.v 的代码如下：

```
module song_reader_ctrl(clk, reset, play, note_done, new_note);
    input clk;
    input reset;
    input play;
    input note_done;
    output reg new_note;

    parameter ST_RESET     = 2'd0;
    parameter ST_NEW_NOTE  = 2'd1;
    parameter ST_WAIT      = 2'd2;
    parameter ST_NEXT_NOTE = 2'd3;

    reg [1:0] current_state, next_state; //现态与次态
    //状态寄存器
    always @(posedge clk or posedge reset) begin
        if (reset)
            current_state <= ST_RESET;     //异步高电平复位
        else
            current_state <= next_state;
```

```
20      end
21
22      //状态转移判断
23      always @(*) begin
24          next_state = current_state;        //赋初值，避免产生锁存器
25          case (current_state)
26              ST_RESET: begin
27                  if (play)
28                      next_state = ST_NEW_NOTE;    //play为高电平时进入
                            NEW_NOTE
29              end
30              ST_NEW_NOTE: begin
31                  next_state = ST_WAIT;            //等待
32              end
33              ST_WAIT: begin
34                  if (!play)
35                      next_state = ST_RESET;       //停止播放，回到RESET
36                  else if (note_done)
37                      next_state = ST_NEXT_NOTE;   //若当前音符播放完毕，则
                            NEXT_NOTE
38              end
39              ST_NEXT_NOTE: begin
40                  next_state = ST_NEW_NOTE;        //延迟一个时钟周期后回到
                        NEW_NOTE
41              end
42              default: next_state = ST_RESET;
43          endcase
44      end
45
46      //输出逻辑
47      always @(*) begin
48          //在ST_NEW_NOTE状态时输出高电平脉冲，其他状态均为低电平
49          if (current_state == ST_NEW_NOTE)
50              new_note = 1'b1;
51          else
52              new_note = 1'b0;
53      end
54  endmodule
```

结束判断模块 song_over.v 的代码如下：

```
module song_over(clk, reset, ci, din, dout);
    input clk;
    input reset;
    input ci;           //ci为地址计数器的进位co，高电平表示乐曲结束
    input [5:0] din;    //即rom输出的duration
    output reg dout;    //即song_done

    //ci为1或din为0时触发结束
    wire trigger = (ci == 1'b1) || (din == 6'b000000);

    //状态编码：空闲等待、发脉冲、等待触发条件消失
    parameter ST_IDLE  = 2'd0;
    parameter ST_PULSE = 2'd1;
    parameter ST_WAIT  = 2'd2;

    reg [1:0] current_state, next_state;

    //状态寄存器
    always @(posedge clk or posedge reset) begin
        if (reset)
            current_state <= ST_IDLE;
        else
            current_state <= next_state;
    end

    //次态判断
    always @(*) begin
        next_state = current_state; //默认保持状态
        case (current_state)
            ST_IDLE: begin
                //当检测到结束条件时，进入发脉冲状态
                if (trigger)
                    next_state = ST_PULSE;
            end
            ST_PULSE: begin
                //发出1个时钟周期脉冲后进入等待状态
                next_state = ST_WAIT;
            end
            ST_WAIT: begin
```

```
                //为防止连续误发脉冲，必须等待触发条件完全消失，才能回到空闲状态
                if (!trigger)
                    next_state = ST_IDLE;
            end
            default: next_state = ST_IDLE;
        endcase
    end

    //输出逻辑
    always @(*) begin
        if (current_state == ST_PULSE)
            dout = 1'b1;  //在ST_PULSE状态时输出高电平
        else
            dout = 1'b0;  //其他状态保持低电平
    end
endmodule
```

歌曲存储模块 song_rom 新增一首歌曲《春日影》，其代码如下：

```
assign memory[  95      ] =     {6'd0, 6'd26}   ;       // Note:  rest
assign memory[  96      ] =     {6'd43, 6'd36}  ;       // Note:  D#6
assign memory[  97      ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  98      ] =     {6'd39, 6'd12}  ;       // Note:  B5
assign memory[  99      ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  100     ] =     {6'd43, 6'd24}  ;       // Note:  D#6
assign memory[  101     ] =     {6'd44, 6'd6}   ;       // Note:  E6
assign memory[  102     ] =     {6'd43, 6'd6}   ;       // Note:  D#6
assign memory[  103     ] =     {6'd41, 6'd36}  ;       // Note:  C#6
assign memory[  104     ] =     {6'd43, 6'd36}  ;       // Note:  D#6
assign memory[  105     ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  106     ] =     {6'd39, 6'd12}  ;       // Note:  B5
assign memory[  107     ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  108     ] =     {6'd43, 6'd24}  ;       // Note:  D#6
assign memory[  109     ] =     {6'd44, 6'd6}   ;       // Note:  E6
assign memory[  110     ] =     {6'd43, 6'd6}   ;       // Note:  D#6
assign memory[  111     ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  112     ] =     {6'd39, 6'd12}  ;       // Note:  B5
assign memory[  113     ] =     {6'd41, 6'd12}  ;       // Note:  C#6
assign memory[  114     ] =     {6'd31, 6'd12}  ;       // Note:  D#5
```

```
21 assign memory[  115    ] =    {6'd31, 6'd12}  ;       // Note:  D#5
22 assign memory[  116    ] =    {6'd29, 6'd12}  ;       // Note:  C#5
23 assign memory[  117    ] =    {6'd32, 6'd12}  ;       // Note:  E5
24 assign memory[  118    ] =    {6'd31, 6'd12}  ;       // Note:  D#5
25 assign memory[  119    ] =    {6'd29, 6'd12}  ;       // Note:  C#5
26 assign memory[  120    ] =    {6'd29, 6'd12}  ;       // Note:  C#5
27 assign memory[  121    ] =    {6'd29, 6'd12}  ;       // Note:  C#5
28 assign memory[  122    ] =    {6'd27, 6'd6}   ;       // Note:  B4
29 assign memory[  123    ] =    {6'd27, 6'd6}   ;       // Note:  B4
30 assign memory[  124    ] =    {6'd32, 6'd12}  ;       // Note:  E5
31 assign memory[  125    ] =    {6'd31, 6'd12}  ;       // Note:  D#5
32 assign memory[  126    ] =    {6'd29, 6'd12}  ;       // Note:  C#5
33 assign memory[  127    ] =    {6'd0, 6'd0}    ;       // Note:  rest
```

3.2 仿真结果

3.2.1 顶层模块及控制器模块

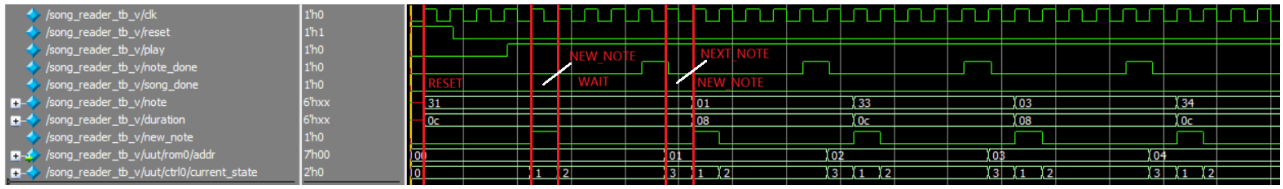


图 20: song_reader 控制器仿真结果

分析：如图中从左往右第一条直线处所示，当第一个时钟上升沿到达时，reset = 1，状态为 RESET（状态编码为 0），此时 play = 0，note_done = 0，new_note = 0；

此后 reset 变为 0，但是由于 play 仍为 0，因此状态保持在 RESET 状态，直到第二条直线处所示的时钟上升沿到达时，reset = 0，play = 1，此时状态转移到 NEW_NOTE 状态（状态编码为 1），此时 play = 1，note_done = 0，new_note = 1；

在第三条直线处，控制器自然进入 WAIT 状态（状态编码为 2），此时 play = 1，note_done = 0，new_note = 0；

在第四条直线处，note_done = 1，当时钟上升沿到达时，状态转移到 NEXT_NOTE 状态（状态编码为 3），此时 play = 1，note_done = 1，new_note = 0；

在第五条直线处，note_done = 0，当时钟上升沿到达时，状态转移到 NEW_NOTE 状态（状态编码为 1），此时 play = 1，note_done = 0，new_note = 1；

综上所述，song_reader 模块的控制器部分仿真结果符合设计要求。

3.2.2 结束判断模块

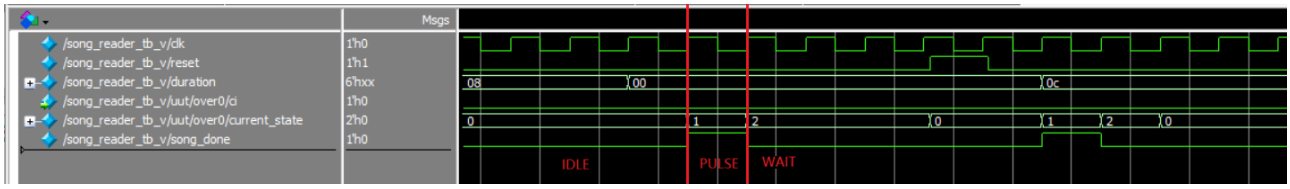


图 21: song_over 仿真结果

分析：如图所示，地址计数器进位输出 ci 始终为 0；一开始结束判断模块处于空闲状态 IDLE （状态编码为 0），在从左往右第一条直线处，duration = 0，当时钟上升沿到达时，状态转移到 PULSE 状态（状态编码为 1），此时输出 song_done = 1；发出一个时钟周期脉冲后进入等待状态 WAIT（状态编码为 2）并挂起，此时输出 song_done 变为 0，如第二条直线处所示。

综上所述，song_over 模块的仿真结果符合设计要求。

3.2.3 控制器、地址计数器与 ROM 的时序关系

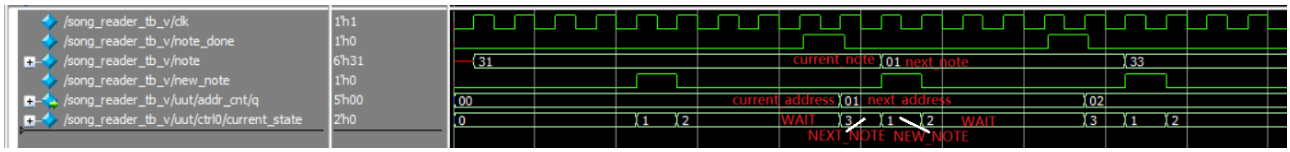


图 22: 控制器、地址计数器与 ROM 的时序关系图

分析：如图所示，song_reader 控制器、地址计数器与 ROM 的时序关系与书中的图片一致，设计符合要求。

4. note_player 模块

4.1 Verilog HDL 代码设计

note_player 模块由顶层模块 note_player.v 和子模块控制器 note_player_ctrl.v、音符节拍定时器 note_timer.v、DDS 模块 dds.v、D 触发器 dffre.v、只读存储器 frequency_rom.v 组成。

顶层模块代码如下：

```
1 module note_player(clk, reset, play_enable, note_to_load,
    duration_to_load, load_new_note, sampling_pulse, beat, note_done,
    sample, sample_ready);
2       input clk;
3    input reset;
```

```
    input play_enable;              //高电平表示播放
    input [5:0] note_to_load;       //需播放的音符标记
    input [5:0] duration_to_load;   //需播放音符的持续时间
    input load_new_note;            //新的音符需播放脉冲
    input sampling_pulse;           //频率48kHz，索取新的正弦样品
    input beat;                     //频率48Hz，定时基准信号

    output note_done;               //音符播放完毕
    output [15:0] sample;           //正弦波样品输出
    output sample_ready;            //下一个正弦波样品就绪

    wire [5:0] q;               //暂存音符标记
    wire [19:0] dout;           //FreqROM输出的相位增量后20位
    wire timer_clear;           //计时器清零信号
    wire timer_done;            //计时器完成信号
    wire load;                  //音符寄存器使能信号

    wire data_path_reset = reset || (~play_enable);

    //D触发器实例
    dffre #(.n(6)) note_reg (.clk(clk), .en(load), .r(data_path_reset), .
       d(note_to_load), .q(q));
    //frequency_rom实例
    frequency_rom freq_rom (.clk(clk), .addr(q), .dout(dout));
    //DDS实例
    dds dds_inst (.clk(clk), .reset(data_path_reset), .sampling_pulse(
       sampling_pulse), .k({2'b00, dout}), .sample(sample), .
       new_sample_ready(sample_ready));
    //控制器实例
    note_player_ctrl ctrl_inst (.clk(clk), .reset(reset), .play_enable(
       play_enable), .load_new_note(load_new_note), .timer_done(
       timer_done), .load(load), .timer_clear(timer_clear), .note_done(
       note_done));
    //音符节拍定时器实例
    note_timer timer_inst (.clk(clk), .beat(beat), .duration_to_load(
       duration_to_load), .timer_clear(timer_clear), .timer_done(
       timer_done));
endmodule
```

控制器模块 note_player_ctrl.v 的代码如下：

```
module note_player_ctrl(clk, reset, play_enable, load_new_note,
   timer_done, load, timer_clear, note_done);
    input clk;
    input reset;
    input play_enable;       //来自mcu, 高电平表示播放
    input load_new_note;     //来自song_reader, 表示索取新音符
    input timer_done;        //来自note_timer, 表示当前音符计时结束
    output reg load;         //输出给dffre, 使能读取新音符
    output reg timer_clear; //输出给note_timer, 清零计时器
    output reg note_done;    //输出给song_reader, 表示当前音符播放完毕

    //状态编码
    parameter ST_RESET = 2'd0;
    parameter ST_WAIT  = 2'd1;
    parameter ST_DONE  = 2'd2;
    parameter ST_LOAD  = 2'd3;

    reg [1:0] current_state, next_state;

    //状态寄存器
    always @(posedge clk or posedge reset) begin
        if (reset)
            current_state <= ST_RESET;
        else
            current_state <= next_state;
    end

    //次态逻辑
    always @(*) begin
        next_state = current_state;      //默认保持现态
        case (current_state)
            ST_RESET: begin
                next_state = ST_WAIT;    //复位后进入WAIT
            end
            ST_WAIT: begin
                if (!play_enable)
                    next_state = ST_RESET;    //未播放或暂停时, 回到RESET
                else if (timer_done)
                    next_state = ST_DONE;     //当前音符播放时长已满
```

```
                else if (load_new_note)
                    next_state = ST_LOAD;     //收到装载新音符的请求
            end
            ST_DONE: begin
                next_state = ST_WAIT; //发出note_done脉冲后回到WAIT
            end
            ST_LOAD: begin
                next_state = ST_WAIT; //发出load脉冲后回到WAIT
            end
            default: next_state = ST_RESET;
        endcase
    end

    //输出逻辑
    always @(*) begin
        //赋初值，避免产生锁存器
        timer_clear = 1'b0;
        load        = 1'b0;
        note_done   = 1'b0;

        case (current_state)
            ST_RESET: begin
                timer_clear = 1'b1;
            end
            ST_WAIT: begin
                //WAIT状态下所有输出均为0
            end
            ST_DONE: begin
                timer_clear = 1'b1;
                note_done   = 1'b1;
            end
            ST_LOAD: begin
                timer_clear = 1'b1;
                load        = 1'b1;
            end
        endcase
    end
endmodule
```

音符节拍定时器模块 note_timer.v 的代码如下：

```
module note_timer(clk, beat, duration_to_load, timer_clear, timer_done);
    input clk;
    input beat;                     //定时基准信号，频率48Hz
    input [5:0] duration_to_load; //当前音符的持续时间
    input timer_clear;              //控制器发来的清零信号
    output timer_done;              //计时结束信号

    reg [5:0] cnt;

    //初始化计数器
    initial cnt = 6'd0;
    //当计数器达到duration_to_load - 1时结束,同时要求duration_to_load > 0
       , 防止duration为0时下溢导致卡死
    assign timer_done = (duration_to_load > 0) && (cnt >= (
       duration_to_load - 1));
    always @(posedge clk) begin
        if (timer_clear)
            cnt <= 6'd0; //同步清零
        else if (beat)
            cnt <= cnt + 1'b1;
    end
endmodule
```

4.2 仿真结果

4.2.1 顶层模块

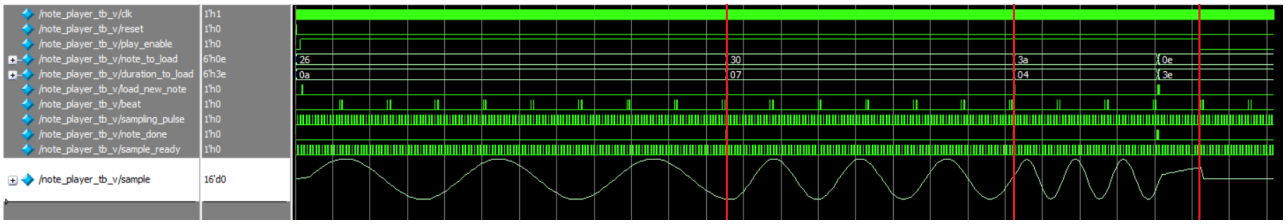


图 23: note_player 仿真结果

分析：如图所示，在从左往右第一、二条直线标注处，note_to_load 增大，输出正弦信号 sample 的频率也随之增大，note_to_load 发生变化的点即为频率发生变化的转折点。在第三条直线处，play_enable = 0 时，note_player 模块处于复位状态，输出 sample = 0，与书上的图片一致，设计符合要求。

4.2.2 note_player 控制器模块

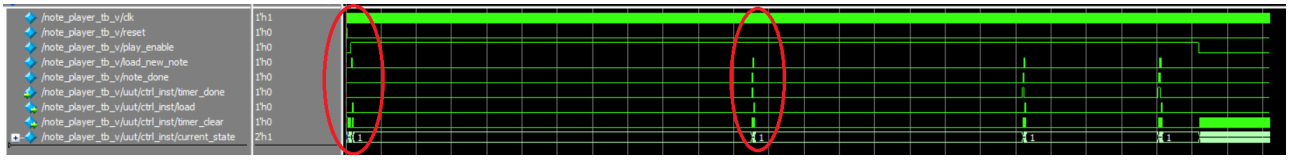


图 24: note_player 控制器仿真结果（全图）

前后两处圆圈圈出部分的局部放大图如下所示：

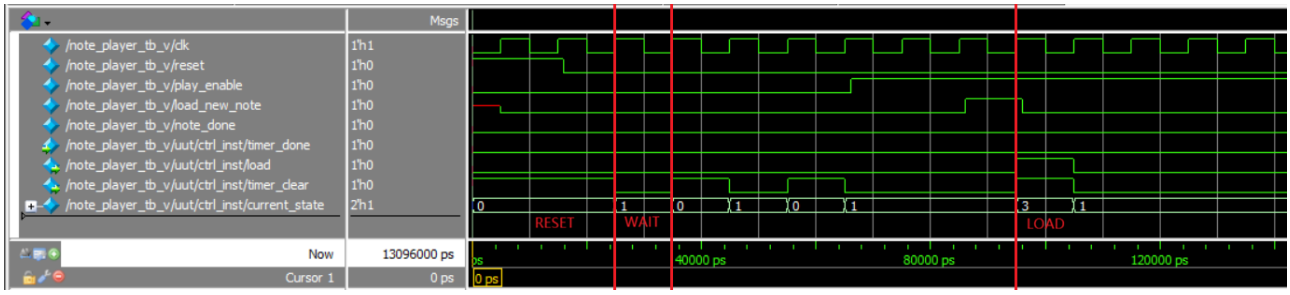


图 25: note_player 控制器仿真结果局部放大图（前）

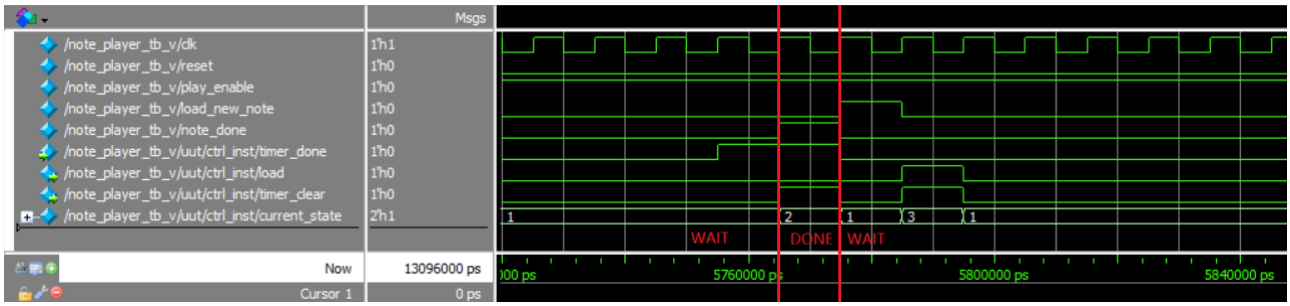


图 26: note_player 控制器仿真结果局部放大图（后）

分析：如图 25 所示，初始状态为 RESET（状态编码为 0），此时 reset = 1，play_enable = 0，load_new_note = 0，timer_done = 0，load = 0，timer_clear = 1，note_done = 0；

在图示从左往右第一条直线处，reset = 0 时，时钟上升沿到达后，状态转移到 WAIT（状态编码为 1），此时 reset = 0，play_enable = 0，load_new_note = 0，timer_done = 0，load = 0，timer_clear = 0，note_done = 0；

在第二条直线处，由于 play_enable = 0，控制器回到 RESET 状态

在第三条直线处，play_enable = 1，load_new_note = 1，当时钟上升沿到达时，状态转移到 LOAD 状态（状态编码为 3），此时 reset = 0，timer_done = 0，load = 1，timer_clear = 1，note_done = 0；

如图 26 所示，在从左往右第一条直线处，前一刻控制器状态为 WAIT，此时 play_enable = 1，timer_done = 1，当时钟上升沿到达时，状态转移到 DONE 状态（状态编码为 2），此时 reset = 0，load_new_note = 0，load = 0，timer_clear = 1，note_done = 1；

在第二条直线处，控制器自然转移到 WAIT 状态。

综上所述，note_player 模块的控制器模块符合设计要求。

4.2.3 音符节拍定时器模块

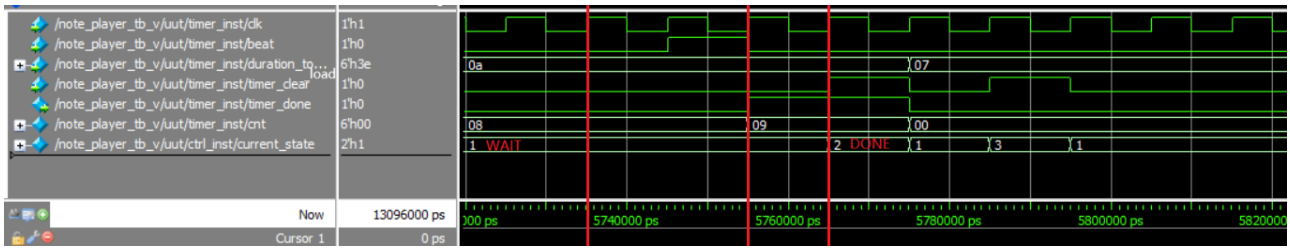


图 27: note_timer 仿真结果局部放大图

分析：如图所示，在从左往右第一条直线处，beat = 0，定时器不计数，cnt 保持为 8；在第二条直线处，beat = 1，定时器开始计数，cnt 从 8 增加到 9，由于此时 duration_to_load = 10，所以当 cnt 达到 9 时，timer_done = 1；在第三条直线处，timer_clear = 1，计数器被清零，cnt 变为 0，timer_done 变为 0。

综上所述，note_timer 模块符合设计要求。

5. 1000 分频器模块

5.1 Verilog HDL 代码设计

1000 分频器即模 1000 的计数器，其状态位数为 10 位，因此只需将 counter_n 模块的两个参数 counter_bits 与 n 设置为 10 即可。其对应测试代码可在课程文件中 lab7_timer12s 的 src 文件夹中给出的测试代码的基础上修改得到。

5.2 仿真结果

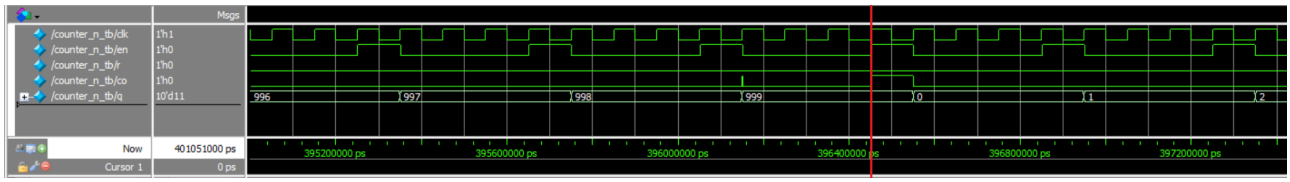


图 28: 1000 分频器仿真结果

分析：如图所示，当使能信号 en = 1 时，分频器正常计数；当计数器计数到 999 后，当 en = 1 时，计数值回到 0，并且进位输出 co = 1，符合模 1000 的计数要求。

6. 同步化电路 sync_circuit

6.1 Verilog HDL 代码设计

根据原理图，同步化电路由两个 D 触发器组成，其代码如下所示：

```
module sync_circuit(in, clk, out);
        input in, clk;
        output out;
        reg q1, q2; //两个触发器的输出

    //初始化寄存器
    initial begin
        q1 = 1'b0;
        q2 = 1'b0;
    end

    always @(posedge clk) begin
        q1 <= in;
        q2 <= q1;
    end

    assign out = q1 & (~q2);
endmodule
```

6.2 仿真结果

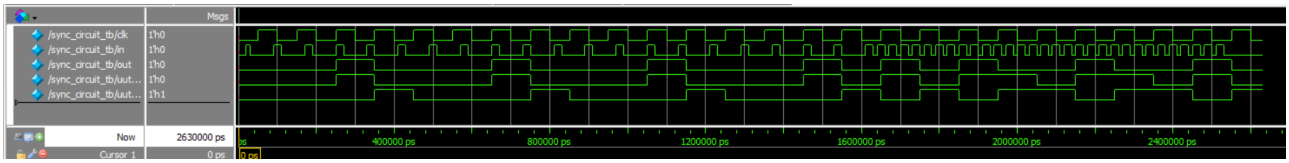


图 29: 同步化电路仿真结果

分析：如图所示，经过一段时间后，在输入 in = 1 的每次时钟上升沿到达时，out 都会产生一个宽度为一个时钟周期的高电平脉冲，符合同步化电路的设计要求。

7. 次顶层模块 music_player

7.1 Verilog HDL 代码设计

music_player 模块由子模块 mcu.v、song_reader.v、note_player.v、sync_circuit.v 和节拍基准产生器 counter_n 组成。

其代码如下所示：

```
//端口列表参考书上要求的写法
module music_player #(parameter sim = 0)(
    input clk,                      //高电平有效
    input reset,                    //复位输入，高电平有效
    input play_pause,               //“播放/暂停”输入，高电平同步脉冲
    input next,                     //“下一首”输入，高电平同步脉冲
    input NewFrame,                 //高电平非同步脉冲，索取新的样品
    output [15:0] sample,           //正弦样品输出
    output play,                    //播放状态指示
    output [1:0] song               //曲号指示
);

    wire reset_play;                //mcu控制器发出的复位脉冲
    wire song_done;                 //乐曲播放结束信号
    wire note_done;                 //单个音符播放结束信号
    wire new_note;                  //索取新音符信号
    wire [5:0] note;                //音符标记
    wire [5:0] duration;            //音符持续时间

    wire ready;                     //同步化后的样品索取脉冲
    wire beat;                      //节拍基准脉冲(48Hz)
    wire sample_ready;              //DDS计算完成输出

    //同步化电路实例
    sync_circuit sync_inst (.clk(clk), .in(NewFrame), .out(ready));
    //分频器实例，仿真模式(sim=1)时分频比为64，实际上板(sim=0)时，分频比
        为1000
    counter_n #(
        .n(sim ? 64 : 1000),
        .counter_bits(sim ? 6 : 10)
    ) beat_gen (
        .clk(clk),
        .en(ready), //以ready脉冲作为使能端
        .r(1'b0),
        .q(),
        .co(beat)
    );
    //主控制器mcu实例
    mcu mcu_inst (.clk(clk), .reset(reset), .play_pause(play_pause), .
```

```
        next(next), .song_done(song_done), .play(play), .reset_play(
            reset_play), .song(song));
39      //乐曲读取模块song_reader实例
40      song_reader reader_inst (.clk(clk), .reset(reset_play), .play(play),
            .song(song), .note_done(note_done), .song_done(song_done), .note(
            note), .duration(duration), .new_note(new_note));
41      //音符播放模块note_player实例
42      note_player player_inst (.clk(clk), .reset(reset_play), .play_enable(
            play), .note_to_load(note), .duration_to_load(duration), .
            load_new_note(new_note), .sampling_pulse(ready), .beat(beat), .
            note_done(note_done), .sample(sample), .sample_ready(sample_ready)
            );
43  endmodule
```

7.2 仿真结果

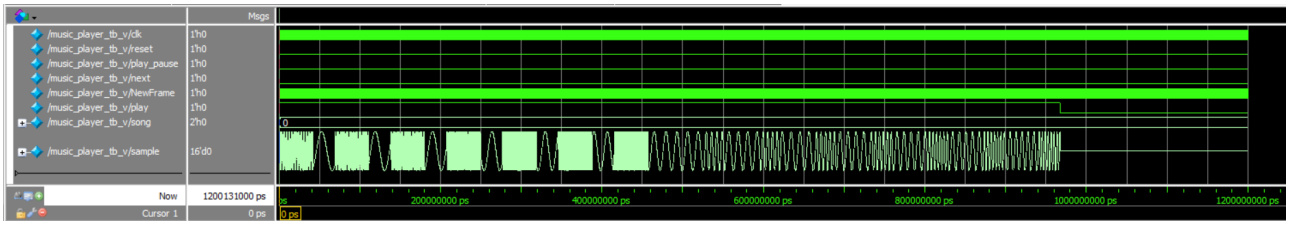


图 30: music_player 仿真结果

分析：如图所示，仿真结果与书上的图片一致，符合设计要求。

8. vivado 工程综合

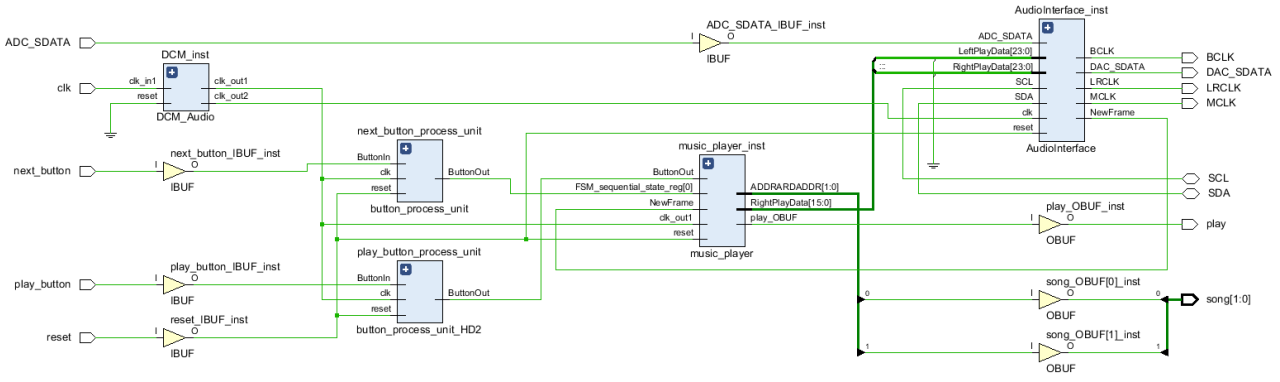


图 31: 综合后的电路原理图

七、思考题

1. 在实验中，为什么 next_button、play_pause_button 两个按键需要消颤动及同步化处理，而 reset 按键不需要消颤动及同步化处理？

答：因为 reset 为置零信号，按下 reset 按键后，系统置零，此后无论 reset 信号如何抖动，系统都保持在置零状态，不会对系统造成不良影响；而 next_button、play_pause_button 的信号会直接影响系统的状态转移，如果不进行消颤动处理，按键抖动可能会导致多次状态转移，造成系统行为异常；如果不进行同步化处理，按键输入可能会与时钟不同步，导致亚稳态问题，进而引发不可预测的系统行为。若 next 信号输入不稳定，可能会导致多次进入下一首歌曲，引起不确定；若 play_pause 信号输入不稳定，可能会导致频繁切换播放/暂停状态，导致播放断断续续，增加系统的损耗，影响系统的使用效果。

2. 在主控制器（mcu）设计中，是否存在接收不到按键信息？若存在，概率多大？有没必要修改设计？

答：存在。由控制器的 ASM 图可知，从 RESET 到 PAUSE 和从 NEXT 到 PLAY 的状态转移过程中，均没有对 play_pause_button 和 next_button 的输入进行检测，因此如果在这两个状态转移过程中按下相应的按键，控制器将无法接收到按键信息。但是由于状态转移的间隔只有一个时钟周期，接收不到按键信息的概率非常小，且即使发生了这种情况，也只是错过了一次按键输入，并不会对系统造成严重影响，因此没有必要修改设计。

八、心得与体会

1. 实验中遇到的问题和解决方法

① 在对 song_reader 模块进行仿真时，我发现仿真器竟然运行了 72 亿 ns，波形图中的 addr、note 等信号也被拉成了一条直线，并在不断向前运行，这种情况我在之前还没有遇到过，我本来以为是代码写错了，检查了很多遍，但没有发现问题所在，但是当我重新运行仿真之后，发现波形又能正常显示，仿真结果也和预期中一样。后来，我查找了一些资料，并询问了 AI，才了解到这种情况是仿真跑飞了，原因可能是我在仿真停下来之后又重新点了一次“Run -All”按钮，因为 initial 块里的语句已经执行完了，此时只有生成时钟的 always #(delay/2) clk= clk; 还在无限循环工作，而当时钟运行了几亿次之后，内存满了，于是把前面的旧波形删掉了，于是便出现了图中的情况，所以只需要重新启动仿真便可以解决问题了。

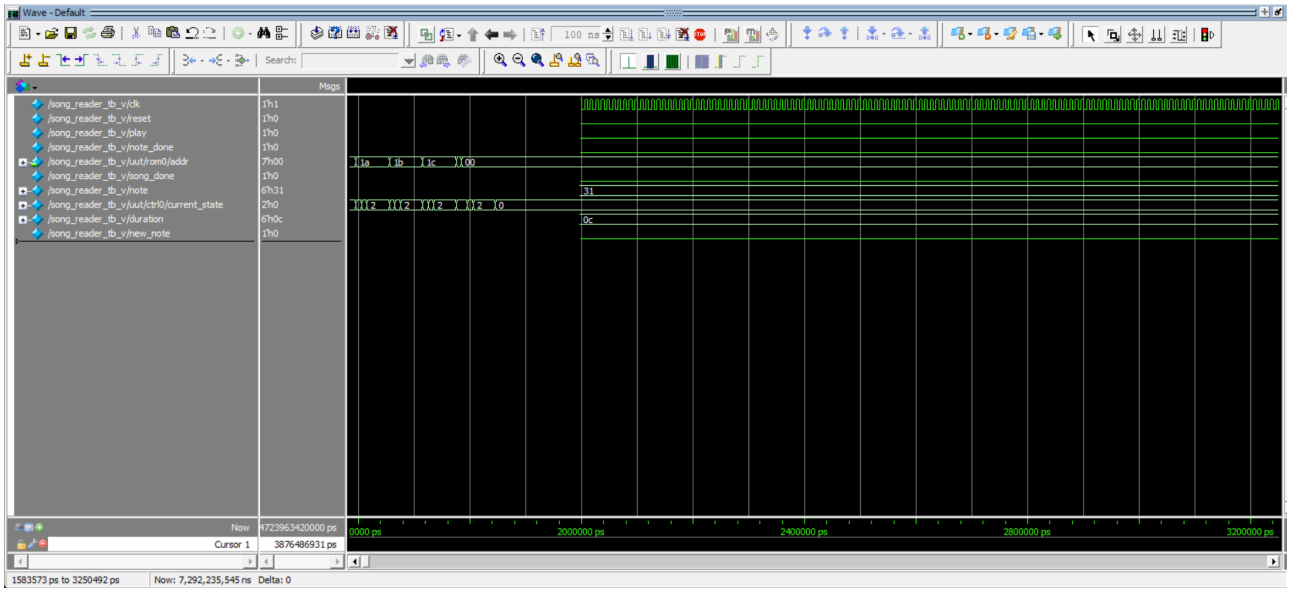


图 32: 仿真错误截图

② 在开发板上测试时，我发现按下 reset 按键之后只有歌曲播放进度归零，歌曲序号没有归零，比如我正在播放第三首歌曲的时候按下 reset，再次按下 play_button，会从第三首歌曲的开头重新开始播放，而不是第一首。于是我回去检查了 mcu 模块的代码，发现在 2 位二进制计数器实例化时的端口连接有误，错误地将 reset 端强行连线到了 0，所以负责按下 reset 按键时，乐曲进度能够正常复位，而负责记录曲号的 2 位二进制计数器并没有收到复位信号 reset ，因此不会回到第一首歌曲。修改代码并重新编译烧录之后便能够正常实现功能了。

错误代码：

```
counter_n #(.n(4), .counter_bits(2)) song_cnt(.clk(clk), .en(nextsong), .r(0), .q(song), .co());
```

修改后：

```
counter_n #(.n(4), .counter_bits(2)) song_cnt(.clk(clk), .en(nextsong), .r(reset), .q(song), .co());
```

2. 心得体会

这次实验的难度和以往都不一样，前面的实验实现的都是一些比较简单的功能，顶层模块代码的编写有示范代码，子模块也都是一些功能简单的模块；但是这次实验要求我们自主设计一个功能较为复杂的工程，老师提供的只有很小一部分的代码，书上也没有可供参考的代码，所以需要我们对着 ASM 图和各个模块的结构框图自己编写代码。一开始，我觉得这么大一个工程，有点无从下手，但是花了点时间阅读书上的原理部分和老师提供的 PPT 之后，便有了初步的思路，先从 lab15 中介绍的 DDS 模块开始设计，然后按照书上推荐的顺序逐步攻

破各个子模块，最后在顶层设计中进行实例化与连线，这样不仅能够使代码的结构清晰明了，而且方便我们后期进行检查调试。在编写 mcu_ctrl、song_reader_ctrl 和 note_player_ctrl 等核心控制模块的代码编写的过程中，我采取了三段式状态机的结构：第一段为同步时序逻辑，专门负责状态寄存器的更新；第二段为纯组合逻辑，专门负责次态的跳转判断；第三段为纯组合逻辑，专门负责信号的输出。这样的结构不仅方便我们阅读和检查，而且能够有效避免意外产生竞争-冒险现象、锁存器（latch）和输出信号的毛刺等问题。一开始写代码的时候没有什么思路，但是直到看了老师在学在浙大上传的视频，跟着里面的内容一步步学，也逐渐摸清了门道，经过了这次实验，我对“自顶而下”的数字系统设计方法有了更深刻的理解，尤其是当我后来学到状态机的理论知识的时候，我对这个实验的设计过程有了更多的体会，当最后开发板上播放出我自己编写为程序的《春日影》时，我越发感觉到数字系统设计的有趣之处，它能够将我们想要实现的目标从软件层面转到硬件层面，从理论层面转到实践层面。